

VANIA PAMELA PATZI QUISPE

1. Análisis del Proyecto Integrador

a. Identificación clara del endpoint

Los endpoints que se consumieron son los siguientes:

- POST/login
El que iniciará sesión y nos dará un token
- GET/film
Este nos dará la lista de todas las películas (requiere inicio de sesión)
- GET /film-stats
Este nos da las estadísticas de películas (requiere inicio de sesión)

El token se guarda temporalmente en memoria (sessionToken) al iniciar sesión.

b. Justificación de la elección del endpoint

Se eligió GraphQL porque permite pedir solo la información que realmente se necesita en una sola solicitud, lo que hace que todo funcione más rápido y evita recibir datos innecesarios, como suele pasar con las APIs REST. Además, se utilizó el endpoint */graphql* porque permite acceder de forma estructurada a la base de datos de películas (*film*) y seleccionar únicamente los campos necesarios, lo cual mejora la eficiencia y da mayor flexibilidad en comparación con las APIs REST tradicionales.

c. Descripción técnica del endpoint (estructura, método, formato de datos)

CARACTERICA	DESCRIPCION
Metodo	POST (login), GET (films), GET (film-stats)
URL	• http://localhost:4000/graphql • http://localhost:4002/films • http://localhost:4002/film-stats
Formato de datos(login)	Json(user, contraseña)
Formato de respuestas	JSON
Autenticacion	JWT

- Se consume el endpoint externo:
 - <http://localhost:4000/graphql>
desde el archivo services.js usando graphql-request.
- También se consume el endpoint de autenticación:
 - <http://localhost:4003/login>
usando axios para obtener un JWT simulado.

2. Diseño del Microservicio

a. Objetivo del microservicio

El microservicio graphql-microservice tiene como objetivo consultar películas desde el backend principal, realizar login previo y procesar estadísticas útiles como promedio de duración, clasificación y características especiales.

b. Elección y justificación de la tecnología de comunicación

VANIA PAMELA PATZI QUISPE

Se eligió GraphQL porque permite consultar múltiples campos específicos en una sola petición, mejorando el rendimiento y evitando el sobrecargado de datos que suelen generar los endpoints REST tradicionales.

c. Diagrama del flujo de integración



3. Implementación Técnica

a. Desarrollo del microservicio funcional

Se implementó con Express, dividido en:

- /login: simula autenticación
- /films: lista de películas

graphql-microservice

├── index.js	# Servidor principal con endpoints: /login, /films, /film-stats
├── services.js	# Lógica de login + consulta a GraphQL (token + datos)
├── package.json	# Dependencias (express, axios, graphql-request, etc.)
└── README.md	# Documentación del microservicio/film-stats: estadísticas

CODIGO BASE

I. Graphql-micriservicio

- Index.js

VANIA PAMELA PATZI QUISPE

```
const express = require("express");
const cors = require("cors");
const { request, gql } = require("graphql-request");
const axios = require("axios");

const app = express();
app.use(cors());
app.use(express.json());

const PORT = 4002;
let sessionToken = null;

app.get("/", (req, res) => {
  res.send(" Bienvenido. Usa POST /login primero.");
});

app.post("/login", async (req, res) => {
  const { username, password } = req.body;

  try {
    const response = await axios.post("http://localhost:4003/login", { username, password });
  } catch (error) {
    sessionToken = null;
    res.status(401).json({ error: "Usuario o contraseña incorrectos" });
  }
});

const requireLogin = (req, res, next) => {
  if (!sessionToken) {
    return res.status(403).json({ error: " Debes iniciar sesión en /login primero" });
  }
  next();
};

const FILMS_QUERY = gql`
query {
  films {
    film_id
    title
    release_year
    length
    rental_rate
    replacement_cost
    rating
  }
}
```

VANIA PAMELA PATZI QUISPE

```
      special_features
    }
  }
};

const getFilms = async () => {
  const response = await request({
    url: "http://localhost:4000/graphql",
    document: FILMS_QUERY,
    requestHeaders: {
      Authorization: `Bearer ${sessionToken}`
    }
  });
  return response.films;
};

app.get("/films", requireLogin, async (req, res) => {
  try {
    const films = await getFilms();
    res.json(films);
  } catch (error) {
    console.error("✖ Error en /films:", error.message);
    res.status(500).json({ error: "Error al obtener películas" });
  }
});

app.get("/film-stats", requireLogin, async (req, res) => {
  try {
    const films = await getFilms();
    if (!Array.isArray(films) || films.length === 0) {
      return res.json({});
    }

    const years = films.map(f => parseInt(f.release_year)).filter(Boolean);
    const lengths = films.map(f => f.length || 0);
    const rentalRates = films.map(f => f.rental_rate || 0);
    const ratings = {};
    const features = {};

    films.forEach(f => {
      // Contador por clasificación
      if (f.rating) {
        ratings[f.rating] = (ratings[f.rating] || 0) + 1;
      }
      // Contador por características especiales
      if (f.special_features) {
        f.special_features.split(',').forEach(feats => {
```

VANIA PAMELA PATZI QUISPE

```
const trimmed = feat.trim();
features[trimmed] = (features[trimmed] || 0) + 1;
});
}
});

const total = years.length;
const minYear = Math.min(...years);
const maxYear = Math.max(...years);
const avgYear = parseFloat((years.reduce((a, b) => a + b, 0) / total).toFixed(2));
const avgLength = parseFloat((lengths.reduce((a, b) => a + b, 0) / total).toFixed(2));
const avgRentalRate = parseFloat((rentalRates.reduce((a, b) => a + b, 0) /
total).toFixed(2));

res.json({
  total,
  minYear,
  maxYear,
  avgYear,
  avgLength,
  avgRentalRate,
  ratings,
  features
});

} catch (error) {
  console.error("✖ Error en /film-stats:", error.message);
  res.status(500).json({ error: "Error al obtener estadísticas" });
}
});

app.listen(PORT, () => {
  console.log(` Microservicio corriendo en http://localhost:${PORT} `);
});
```

Services.js

```
const { request, gql } = require("graphql-request");
const axios = require("axios");

const AUTH_URL =
"http://localhost:4003/login";
const GRAPHQL_URL =
"http://localhost:4000/graphql";

// Obtiene el token del microservicio de login
const getToken = async () => {
  const response = await
  axios.post(AUTH_URL, {
    username: "admin",
    password: "admin123"
  });
  return response.data.token;
};
```

VANIA PAMELA PATZI QUISPE

```
// Consulta con todos los campos útiles  
del backend
```

```
const FILMS_QUERY = gql`  
  query {  
    films {  
      film_id  
      title  
      release_year  
      length  
      rental_rate  
      replacement_cost  
      rating  
      special_features  
    }  
  }  
`;
```

II. Auth-microservice

Index.js

```
const express = require("express");  
const cors = require("cors");  
const jwt = require("jsonwebtoken");
```

```
const app = express();  
app.use(cors());  
app.use(express.json());
```

```
const PORT = 4003;  
const SECRET_KEY = "supersecreto";
```

```
app.post("/login", (req, res) => {  
  const { username, password } = req.body;
```

```
  // Simulación de usuario fijo  
  if (username === "admin" && password === "admin123") {  
    const token = jwt.sign({ username }, SECRET_KEY, { expiresIn: "1h" });  
    return res.json({ token });  
  }
```

```
  res.status(401).json({ message: "Credenciales incorrectas" });  
});
```

```
app.listen(PORT, () => {  
  console.log(` Auth Microservicio en http://localhost:${PORT}/login`);  
});
```

Instalar: ***npm install express cors jsonwebtoken***

```
// Llama al endpoint GraphQL usando el  
token
```

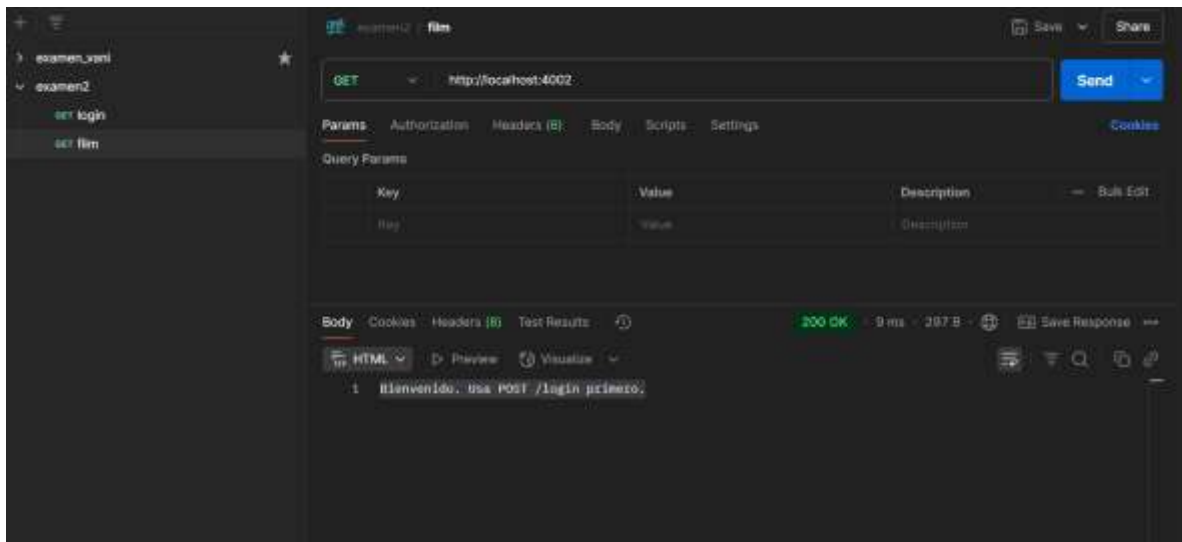
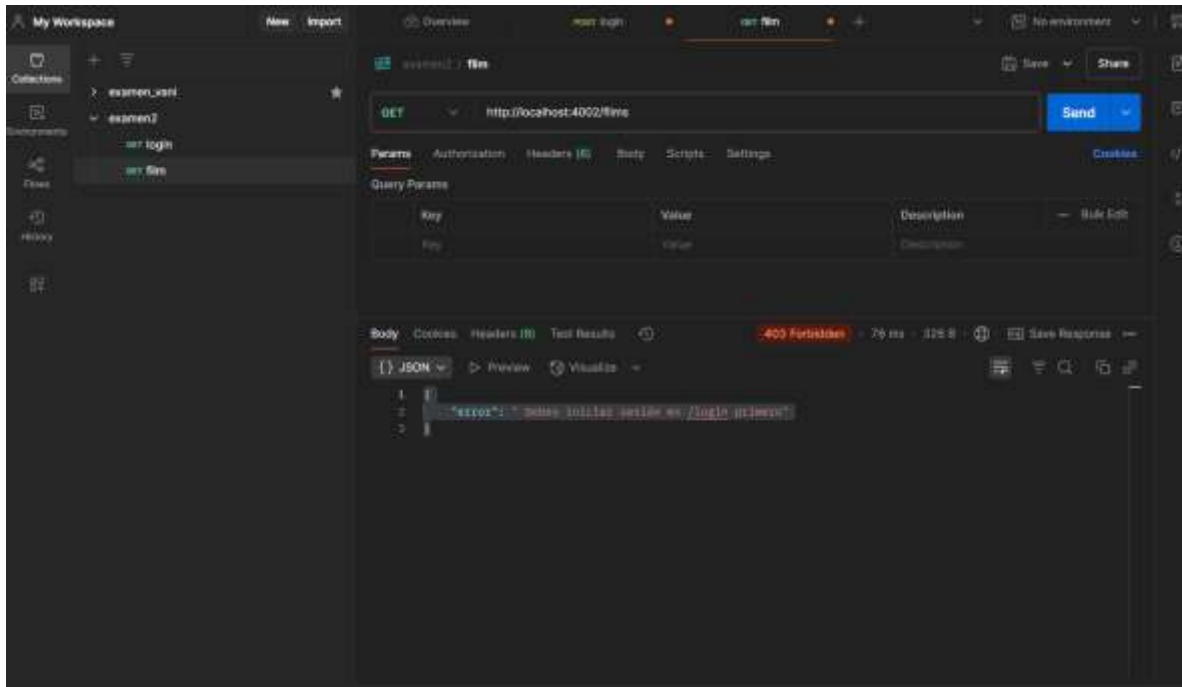
```
const getFilms = async () => {  
  const token = await getToken();  
  const response = await request({  
    url: GRAPHQL_URL,  
    document: FILMS_QUERY,  
    requestHeaders: {  
      Authorization: `Bearer ${token}`  
    }  
  });  
  return response.films;  
};
```

```
module.exports = { getFilms, getToken };
```

VANIA PAMELA PATZI QUISPE

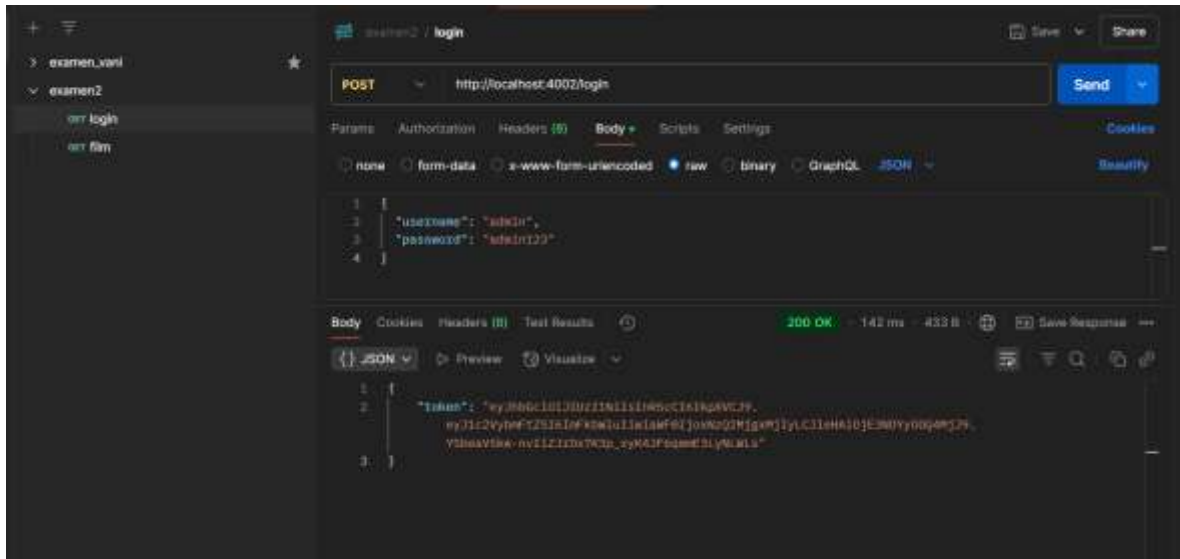
b. Consumo correcto del endpoint externo

Se puede observar lo siguiente:

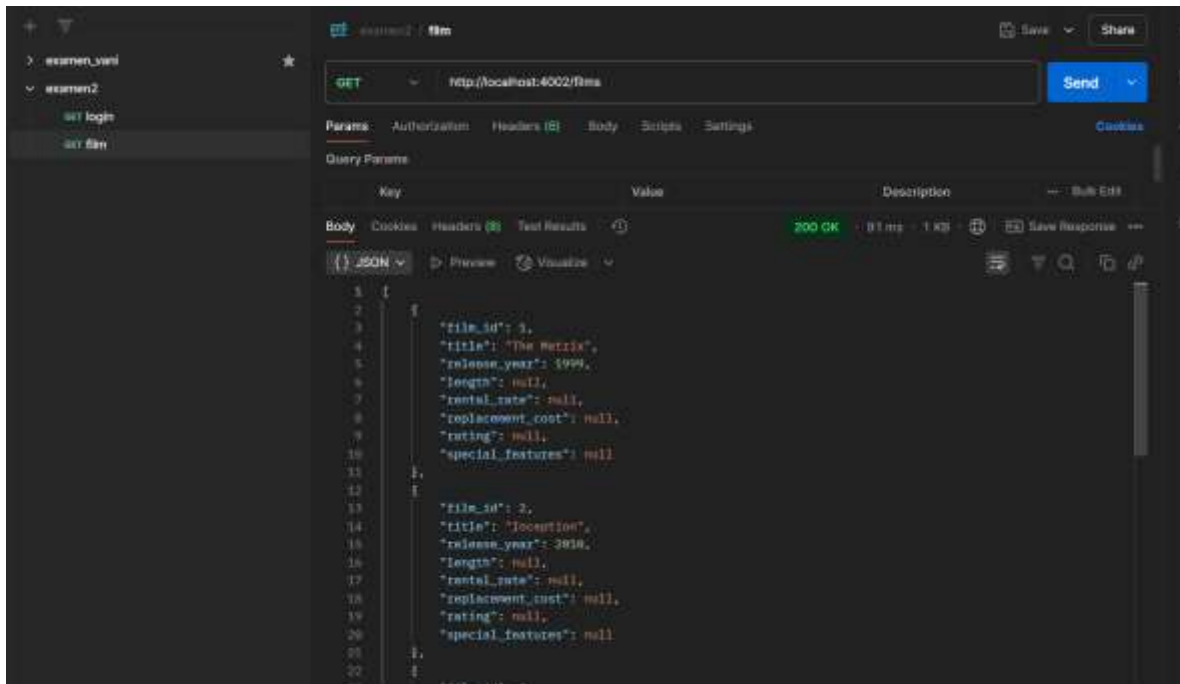


Inicia sesión

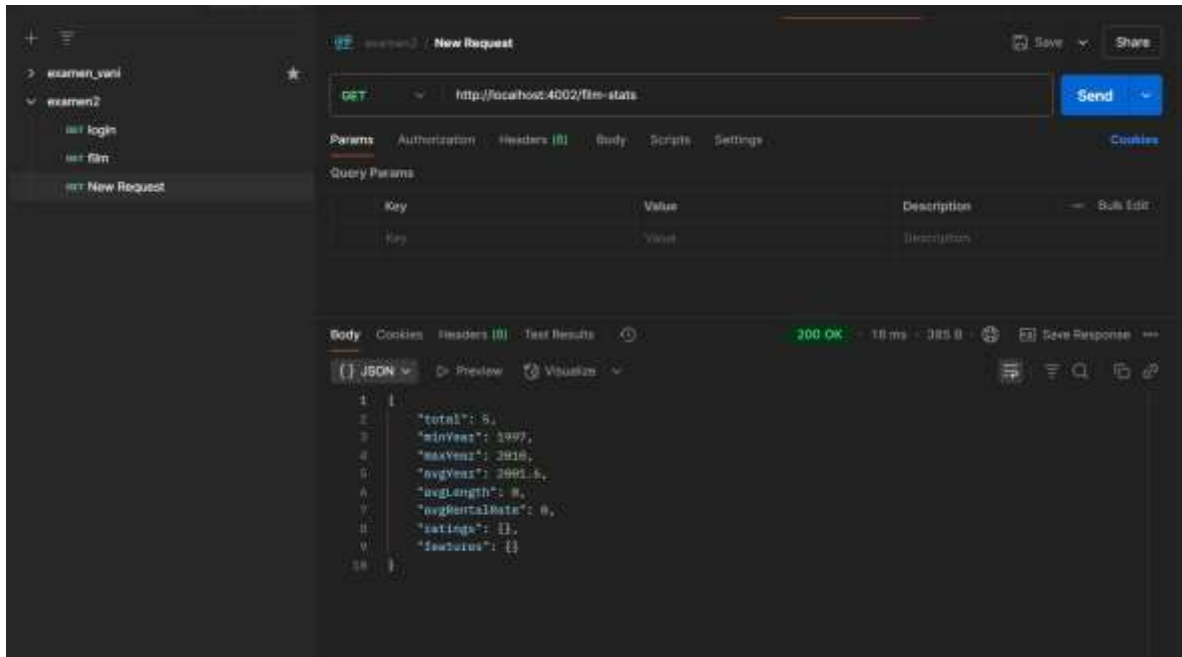
VANIA PAMELA PATZI QUISPE



c. Procesamiento y transformación de los datos



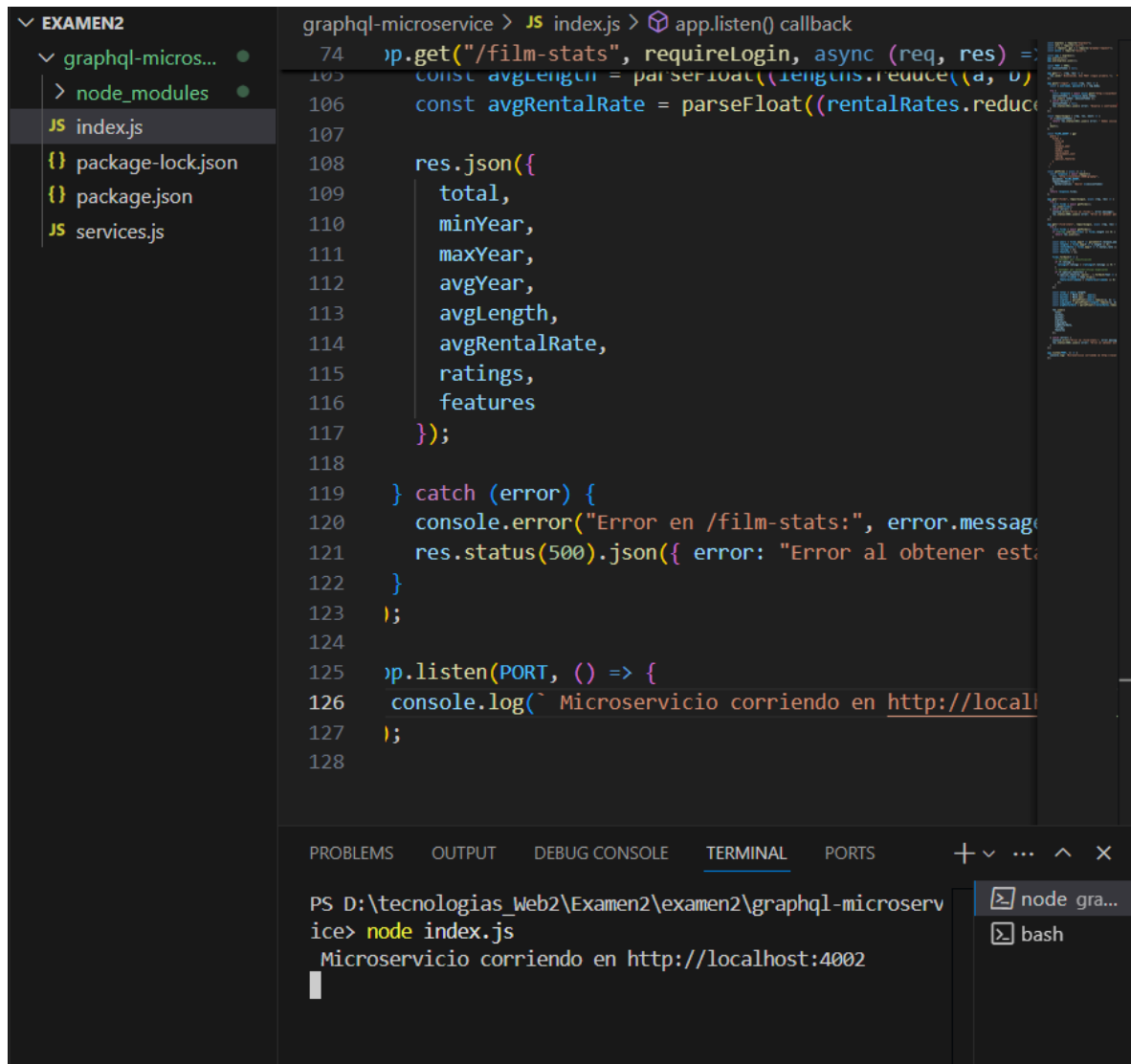
VANIA PAMELA PATZI QUISPE



4. Pruebas y Documentación

Se creo dos microservicios una de login una vez que iniciamos

VANIA PAMELA PATZI QUISPE



The screenshot shows a Visual Studio Code editor with a project named 'EXAMEN2'. The file explorer on the left shows a directory structure for 'graphql-microservice' containing 'node_modules', 'index.js', 'package-lock.json', 'package.json', and 'services.js'. The main editor displays the 'index.js' file, which contains JavaScript code for a GraphQL microservice. The code includes a GET endpoint for '/film-stats' that requires login and returns film statistics. The terminal at the bottom shows the command 'node index.js' being executed, and the output indicates the microservice is running on http://localhost:4002.

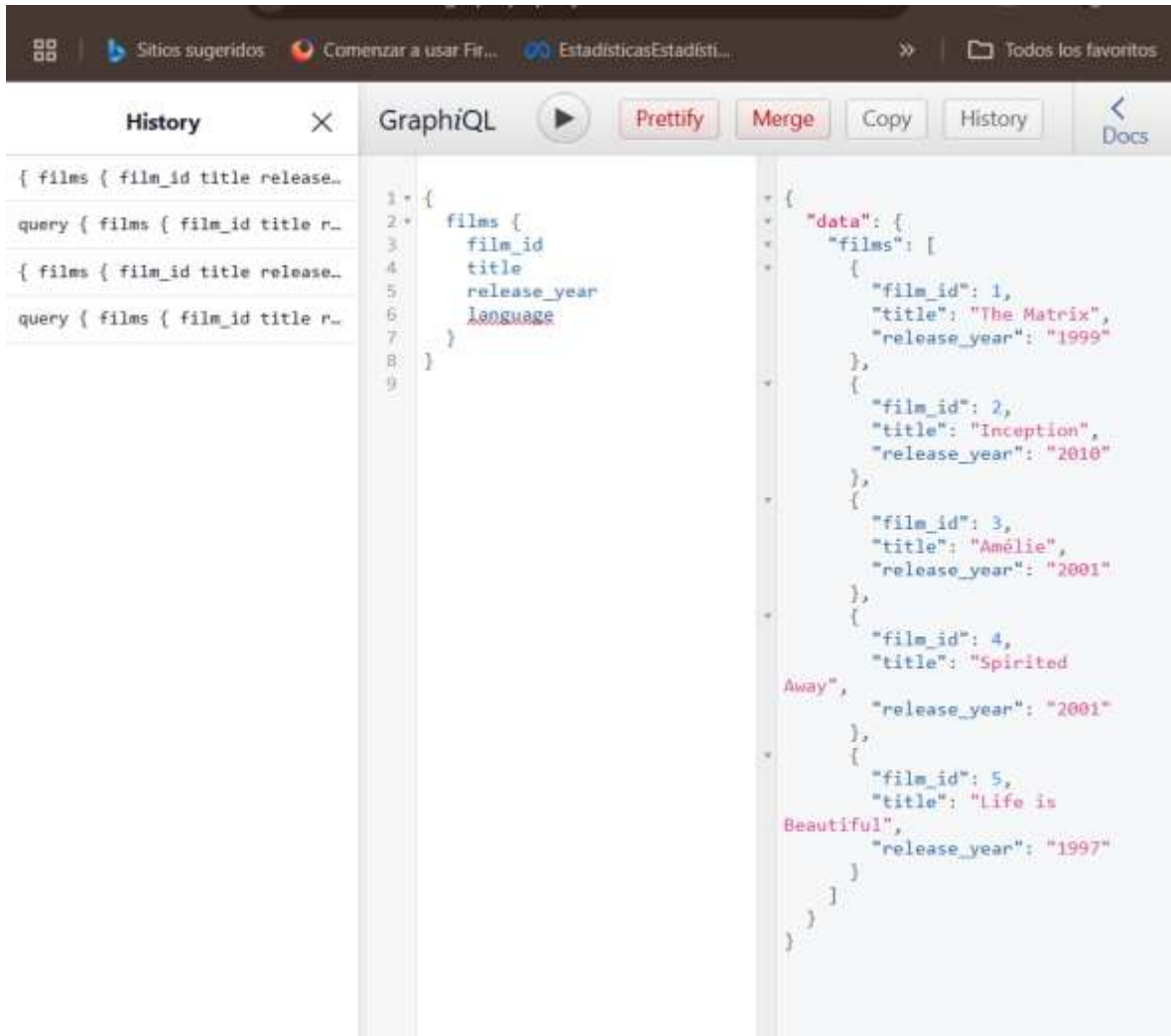
```
graphql-microservice > JS index.js > app.listen() callback
74  app.get("/film-stats", requireLogin, async (req, res) => {
105  const avgLength = parseFloat((lengths.reduce((a, b) => a + b, 0) / lengths.length));
106  const avgRentalRate = parseFloat((rentalRates.reduce((a, b) => a + b, 0) / rentalRates.length));
107
108  res.json({
109    total,
110    minYear,
111    maxYear,
112    avgYear,
113    avgLength,
114    avgRentalRate,
115    ratings,
116    features
117  });
118
119  } catch (error) {
120    console.error("Error en /film-stats:", error.message);
121    res.status(500).json({ error: "Error al obtener esta información" });
122  }
123  });
124
125  app.listen(PORT, () => {
126    console.log(` Microservicio corriendo en http://localhost:${PORT}`);
127  });
128
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS + - ... ^ x

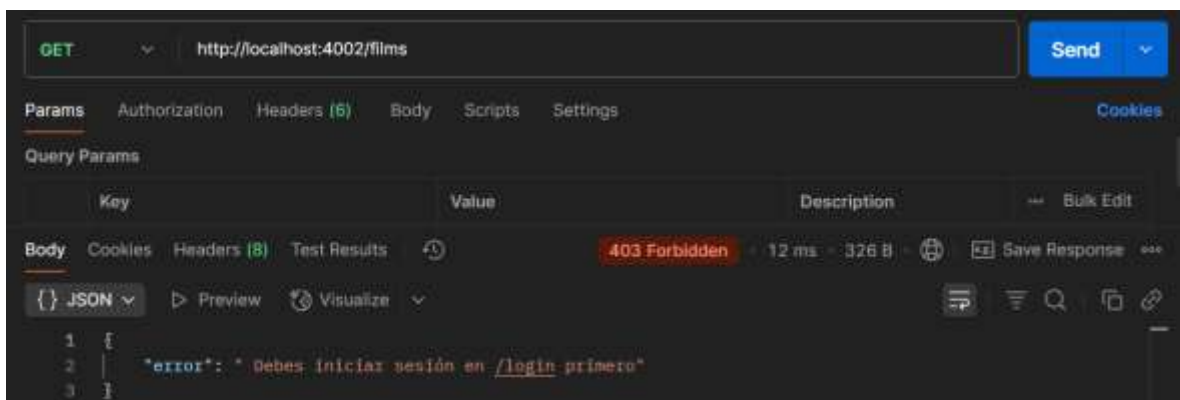
```
PS D:\tecnologias_Web2\Examen2\examen2\graphql-microserv
ice> node index.js
Microservicio corriendo en http://localhost:4002
```

También podemos ver que el de películas esta funcionando:

VANIA PAMELA PATZI QUISPE

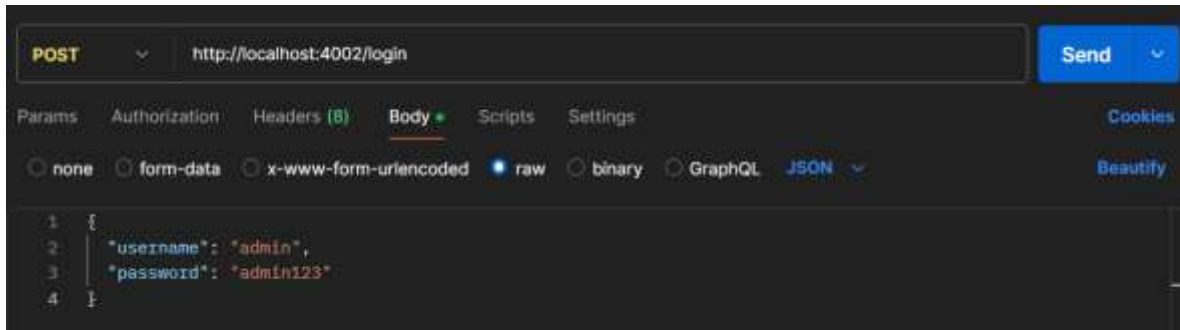


Si no iniciamos sesión nos saldrá un error:



Hay que mandar el json de usuario y contraseña

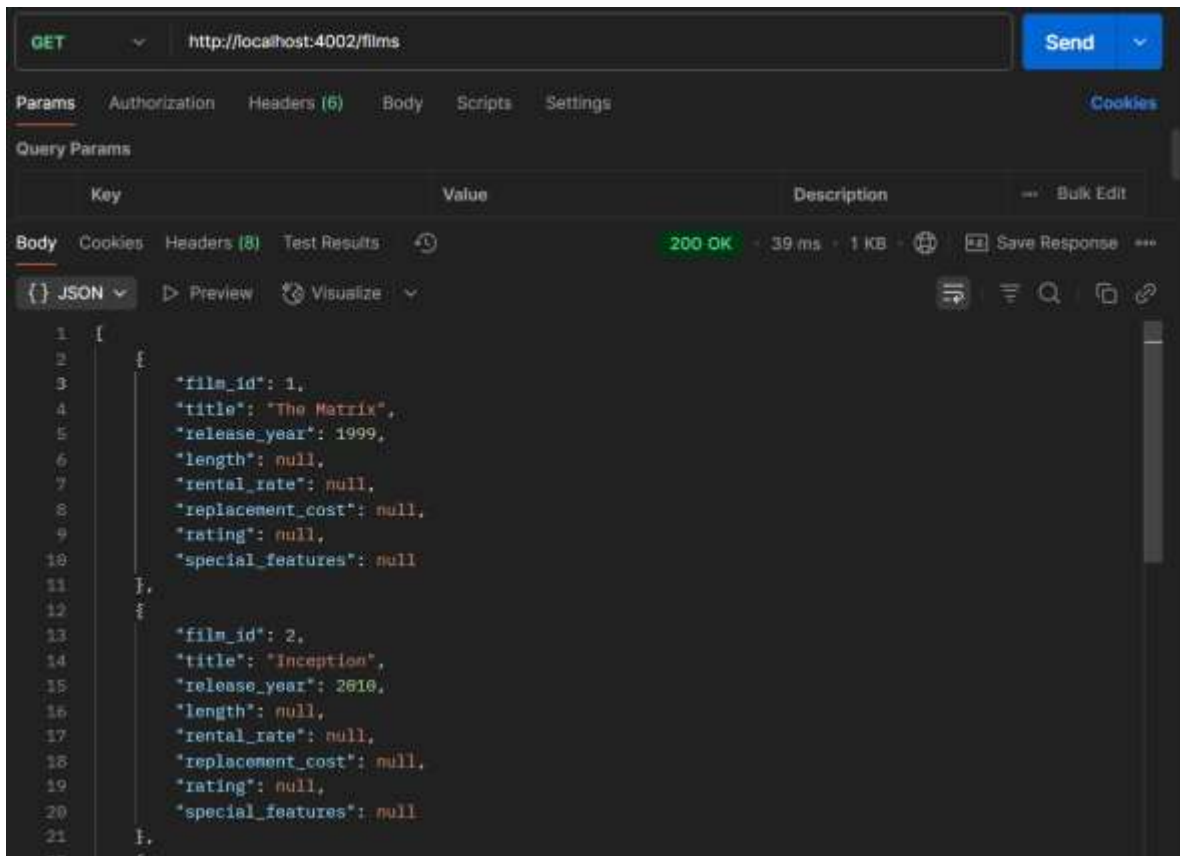
VANIA PAMELA PATZI QUISPE



Una vez enviado se dara un token

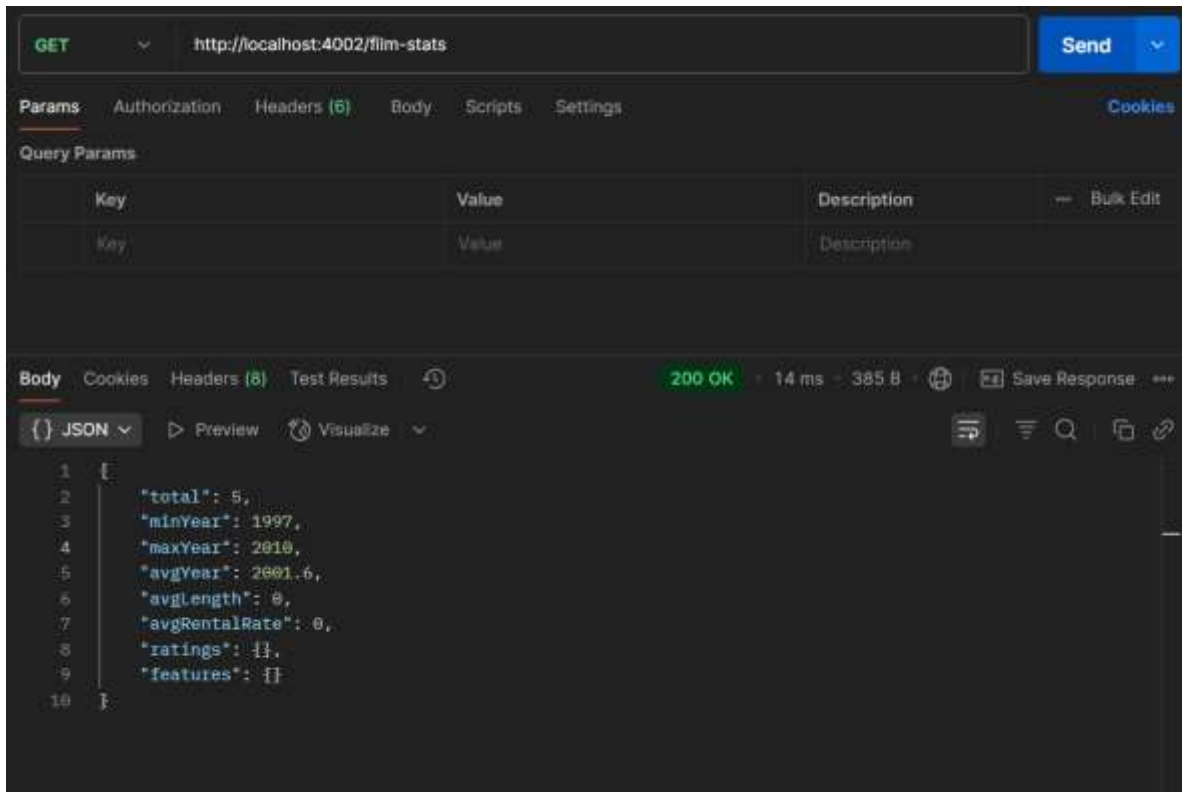


Y asi podremos pasar a ver la lista de películas



VANIA PAMELA PATZI QUISPE

Y así podremos pasar a ver las estadísticas de películas



The screenshot shows a REST client interface with a GET request to `http://localhost:4002/film-stats`. The response is a 200 OK status with a 14 ms response time and 385 B of data. The response body is a JSON object containing film statistics.

Key	Value	Description
Key	Value	Description

```
{
  "total": 5,
  "minYear": 1997,
  "maxYear": 2010,
  "avgYear": 2001.6,
  "avgLength": 0,
  "avgRentalRate": 0,
  "ratings": {},
  "features": {}
}
```